UNDERSTANDING NEURAL NETWORKS

Building an artificial neural network for image recognition, from scratch

Gonçalo Rua Mota

1. Introduction

The goal of this document is to understand, end-to-end, how to make a Neural Network using only Python. The entire process of learning the concept and inserting will be documented thoroughly.

1. Understanding what is an ANN (Artificial Neural Network)

A neural network is a very rough imitation of our brain. The idea behind this is purely pattern recognition, connecting what we call each “neuron” (which are pure values) to other neurons, and it learns by adjusting the strength of its connections based on its mistakes.

2.1. The structure: layers

A neural network is organized into **layers** – groups of neurons that process information in sequence. Every network, regardless of complexity, follows the same basic structure:

Input Layer This is where the network receives raw data. Each neuron in this layer represents one feature of the input. For image-based networks, each neuron corresponds to a single pixel value. A 28×28 image, for example, becomes a flat list of 784 values — one per neuron.

Hidden Layer(s) These are the layers between input and output. They are called "hidden" simply because they are not directly visible from the outside — you don't control what goes in or comes out of them directly. This is where the network learns to detect patterns: edges, shapes, textures at first, and more abstract concepts in deeper layers. A network can have one hidden layer or many — the more layers, the more complex the patterns it can learn. Networks with many hidden layers are what we call *deep* neural networks, which is where the term *deep learning* comes from.

Output Layer The final layer produces the network's answer. Its structure depends on the task. For a digit classification problem (recognizing 0–9), the output layer has 10 neurons — one per class. The neuron with the highest value is the network's prediction.

Note: the neurons in adjacent layers are fully connected — every neuron in one layer connects to every neuron in the next. This type of architecture is called a **fully connected network**, *or* **dense network**.

2. Preprocessing our data

Obviously, our neural network is not able to read or interpret text. All a computer can read is numbers, so our preprocessing will always involve transforming any available data into numbers ready for the computer to take in.

For our emotion detection system, we will follow this process:

1. **Grab the images (and classify them)**

The images we have to feed our network have to be classified. This means; if we want our network to recognize what Sad looks like, we have to feed it an image of a Sad person and tell the network it is a Sad person.
For this project, we will have the images split in folders (Sad, Happy, Angry, Fear, Surprise).

2. **Pre-processing**

Pre-processing the data is an essential step, doing this we get rid of mostly useless clunk information and we make the entire neural network lighter and faster to train. For this project we will be doing two things as pre-processing: greyscaling and downsizing.

3. Layers, Weights and Back Propagation

We recognize an apple by its features; we know an apple usually has round edges, it has a stem, and a weirdly shaped bottom. By throwing a bunch of images of grayscale apples (because the color is very variant and may hinder in the training of recognizing if it's an apple), we have the model to know it's looking at an apple by pattern recognition.

Our brain acts this way; and this is the exact method we use to create a neural network.

1. Layers

A neural network is composed of three layers, each serving their

1.1. Input Layer

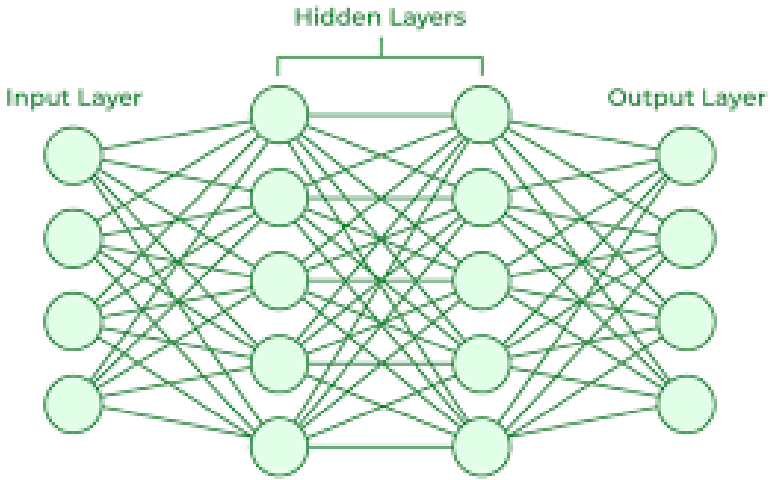
In this layer is where the initial information is located; whether it be raw data, pixels, or anything else. This data has to be numbers.

1.2. Hidden Layers

Inside each neuron, a weight and bias is applied. A weight is a number that says "How much does this input matter to me?"
If a neuron tries to detect a horizontal edge, it needs pixels that form a horizontal line to matter a lot. So the weights for those will be high, while for the others low.
Every neuron in a layer learns a different pattern, because they all start with random weights and get pushed in different directions during training.
So a hidden layer isn't just applying weights, it's basically a collection of neurons each asking a different question about the input.

1.3. Output Layer

Outputs the probability of how right the neural network might be.



2. Weighted Sum

The weighted sum is scoped to each neuron — it measures how strongly that neuron's pattern appears in the input. With 1024 incoming pixel values, the neuron needs to collapse all of that into one number: 'how present is my pattern here?

This matters because each neuron learns a different pattern. Across an entire network, those patterns stack up — edges, shapes, facial features. Together they give the network enough information to recognize what it's looking at.

So the **weighted sum**, of all pixels, is:

$$w_1 \cdot x_1 + w_2 \cdot x_2 \ldots + bias$$

(Each pixel and each of its weights multiplied and summing the bias in the end)

The initialization for these are completely random. This is because we want each neuron to learn different patterns, if we start them all with the same number, they'll all learn the exact same pattern.
If all weights are identical, they will all compute the same outputs, which means identical error signals, identical gradients… etc.

3. What is the bias?

The bias is a single number added to the weighted sum at the end, it has no input, not multiplied by anything. It shifts the result up or down.
Think of it like "I decide what the level of activation before looking at the inputs themselves"

Why do we need a bias?

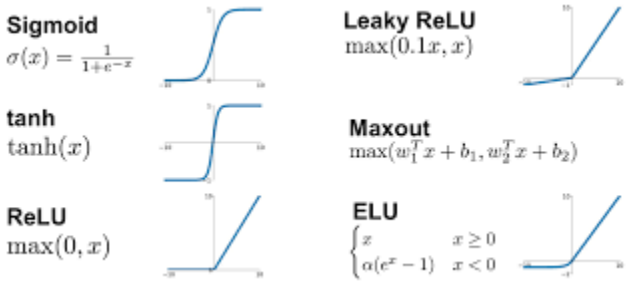
Real world analogy: Think of it like an alarm, the weights decide "How loud does the sound need to be", the bias decides "What's the minimum volume before I bother listening"

Why does it start random?

If every neuron's bias starts at zero, or any other equal value, they'll all have the same threshold. Random biases ensure each neuron has a different threshold from the beginning, guaranteeing neurons divergence in a different direction

4. Activation Functions

An activation function is a simple, mathematical function. It is applied after every weighted sum in every layer.



Why do we need it?

Without an activation function, the neuron will output whatever the sum is. Could be 10,000, -5,000… raw numbers, flowing through the network cause instability.

Why don't we just use raw numbers?

The raw numbers, in themselves, carry more information. The issue is that it's useless information. If we have exploding values, when we adjust our weights in the future, it will massively overshoot when we nudge, never settling.

Additionally, without activation functions, stacking layers is mathematically pointless — every layer is just multiplication and addition, which collapses into a single multiplication and addition regardless of how many layers you have. The network loses all depth.

Isn't adding + multiplying what we do either way?

Your network needs to learn "if eyebrows point down AND mouth points down → angry."

That's an AND relationship — two things must be true simultaneously. A straight line relationship can't capture that. It can only say "more of this input = more likely angry" in a straight proportional way.

Activation functions let neurons say "I only fire if my input crosses a threshold" — which is what allows the network to capture these combined, conditional relationships between features.

Basically, it makes it so each neuron makes it's own decision; not just be a compound of what came before.

5. Activation Functions pt. 2

ReLU `max(0, x)` If the value is negative, output 0. If positive, keep it. Simple and fast. The go-to for hidden layers because it doesn't squash positive values, so gradients flow cleanly during backprop. Downside: neurons can "die" if they always receive negative input and permanently output zero.

Sigmoid `1 / (1 + e^-x)` Squashes any value into 0–1. Useful when you need a probability for a single yes/no decision. Problem: for very large or very small inputs, the curve flattens — gradients become near zero, learning slows down or stops. Called the vanishing gradient problem.

Tanh `tanh(x)` Similar to sigmoid but squashes into -1 to 1. Centered at zero, which makes gradients behave better than sigmoid. Still suffers vanishing gradients at extremes.

Leaky ReLU `max(0.1x, x)` Same as ReLU but instead of zeroing negative values, it lets a tiny fraction through. Fixes the dying neuron problem.

ELU — similar to Leaky ReLU but uses a smooth curve for negatives instead of a straight line. Smoother gradients.

Maxout — learns the activation function itself rather than using a fixed one. Powerful but expensive.

6. Loss Function

We know how wrong each weight is by a **loss function**, and to measure how wrong it is we first have to get the probability itself of what the network is trying to guess.

In this case, let's use an emotion detection system, where the goal is to know if an image is Happy, Sad, or Angry.

Let's start with 3 pixel values, after processing (greyscaling & downscaling) being [0.8, 0.2, 0.5]

[LAYER 1]
Compute each neuron's weighted sum: (random neuron weights), and add a bias (also random at this stage), give it an activation function to keep it normalized.
[LAYER 2]
Repeat the same process with all the neurons in the first layer
[LAYER 3]
Repeat the exact same process (random weights and bias), and in the end, softmax the value obtained (turning logits into probabilities that sum to 1)

FOR EACH LOGIT $e^{(logit\ value)} / sum(e^{(logit\ value)} of\ all\ values)$

THIS COMPUTES THE PROBABILITY OF HOW LIKELY IT IS FOR IT TO BE WHATEVER WE WANT

Let's say, in this case that after we compute all the probabilities:

```
probs = [0.67, 0.24, 0.09]
         happy  sad   angry
```

The real value for this image is happy, y = [1, 0, 0]

Our loss function in this case (for one hot labes) is a cross-entropy loss function, which is none less than:

$$loss = -log(prob)$$

Our prob values are between 0 and 1 ; the -log(prob) makes it so the closer the value is to 0, the higher the value returned will be, and the closer it is to 1, the lower the value will be. This is crucial because it punishes worser values exponentially.

This one in specific is called ***cross-entropy***

7. Back Propagation

The idea of back propagation is to know how much each weight contributed to the loss function.
The network is a chain of operations, every step between a weight and a loss must be accounted for. For this, we start at the loss, compute it's derivative and multiply backwards through every operation using the chain rule, this includes the activation function.
At each point in the network, the backwards signal (of when we're multiplying backwards at each point, neuron) is the derivative of the loss with respect of the output at that point. It's a number that says "if this value increased by a tiny bit, the loss would change this much"
As it travels back through each operation it gets multiplied by the operation's derivative (chain rule) which transforms it into "how much did the thing before this operation contribute to the loss"
The backwards signal passing through the network is the error signal, and what each weight gets from it is its gradient.

ReLU killed a neuron on the forward pass (output was 0). The blame signal arrives at that neuron travelling backwards. What happens to it?

Dying ReLU — if a neuron's weighted sum is permanently negative (often caused by a very negative bias), ReLU kills it every forward pass, the blame signal is multiplied by 0 every backward pass, and that neuron never learns again.

1.10. Gradient

The gradient for a weight tells us two things;

- Direction: should this weight increase or decrease to reduce loss
- Magnitude: how much does this weight changing affects the loss

$$gradient = errorsignal * input$$

The error signal is how much blame arrived at this neuron from the loss, and the input is what passed through this weight on the forward pass.

This is the entire full chain for a weight;

```
1. Forward pass → compute output and loss
2. At output layer → error signal = predicted - true
3. Travel backwards → multiply by each operation's derivative
4. At each weight → gradient = error_signal × input
5. Update → new_weight = old_weight - learning_rate × gradient
```

8. Batches

Having for example, 10.000 images. When do we update the weights?
The choice would be to update every weight after analyzing all 10.000 images or after every single image. This creates a horrible amount of noise.

So we group images into small batches; for example of size 32. We run all 32 images through this network, and then we get a set of 32 predictions and 32 losses. We combine the losses into a single number (usually the average), do a backwards pass on that averaged loss. It's a way to ensure that training isn't too heavily affected by noise.

Neural Network Architecture: CNN

(Convolutional Neural Network)

For this project, we will use a CNN (Convolutional Neural Network); the steps mentioned above work for most neural networks architectures, but they are the steps of a MLP (Multi Layer Perceptron)

1. The problem with MLP on images:

The architecture we described so far is the same used for all neural networks, specially MLP (Multi-Layer-Perceptron), however

When you flatten a 32x32 image and feed into an MLP, each pixel gets a dedicated weight connecting it to each neuron. The network learns that "pixel 47 being bright matters", but it has no concept of where pixel 47 is relative to its neighbors.

This means that the network adapts to exact photos and does not recognize patterns such as edges, curves and textures, which means that it won't be able to recognize patterns in photos and recognize the photos themselves.

2. The filter idea

The CNN, instead of having a separate weight for each pixel, has a small set of weights (called a **filter** or **kernel**) that gets slid across the entire image. This filter (for example, a 3x3 filter), starts at the top left corner of the screen, and gets slid across the entire image, always moving one pixel to the right through the entire row, then down one.

For each 3x3 grid of weights, you compute a weighted sum of those 9 pixels, write down the result, slide the window one pixel to the right (same 9 weights, new pixels). At the end, a whole new 2D Grid of numbers will be formed called a **feature map.**

3. Key insight

Because the same weights are used everywhere, the filter essentially asks the same question at every position; "does this pattern exist here?" (which is exactly what we need)

This property is called *translational invariance*.

4. Pooling

With 32 feature maps the amount of data drastically increases, since we don't need pixel-perfect spatial precision (a rough pattern is enough), we have to solve it by pooling. It involves dividing the feature map into small non-overlapping windows (typically 2x2) and only keep the maximum value from each window.

A 32x32 feature map becomes a 16x16, throwing away redundant detail and halving the dimensions. This also translates into translational invariance; if the edge shifts by one pixel the max in the 2x2 window likely stays the same.

5. Stacking layers

After the first conv layer + pool, the second conv layer, the second conv layer will not operate on raw pixels; it will operate on detecting the patterns within patterns. Early layers detect things such as edges and corners, deeper layers combine those into shapes, shapes into parts, parts into faces, and into full expressions.

6. Flatenning

After the final pooling layer, the 2D feature maps are unrolled into a single flat list of numbers. For example, 32 feature maps of size 16x16 become a single list of 8,192 numbers. This flat list feeds into a fully connected layer which outputs the final logits. During backprop, the error signal simply gets reshaped back into the 2D grid — no math involved, just a reorganization.

7. Max Pooling (backward pass)

On the forward pass, each 2x2 window kept only the maximum value and discarded the other 3. During backprop, all the blame goes to whichever value was the maximum — it was the only one that influenced the output. The other 3 receive a gradient of 0 since they had no effect on the forward pass.

```
image → conv layer → pooling → conv layer → pooling → flatten → fully connected →
softmax → probabilities
```

8. Back Propagation

Back propagation, as we've seen before, just runs the pipeline but backwards.

```
probabilities → softmax → fully connected → flatten → pooling → conv layer → pooling
→ conv layer → image
```

At each step, two things happen;

- Compute the gradients for that layer's weights
- Pass the error signal further backward

Let's start from the end, we have our probabilities, having computed the loss. Now the error signal flows backward.

First thing that happens

We have softmaxed output, we have 3 probabilities 1. [0.7, 0.2, 0.1], the true label for this image is 2.[1, 0, 0]
The error signal is [1] - [2], simple. This is the starting blame, "you were off by this much for this class"

So now, flowing backwards, we need the gradient for each weight which'll be;

$$gradient = input \cdot blame\ (or\ error\ signal)$$

And the error signal to pass backwards;

$$errorsignal = weight \cdot blame\ (initial\ error\ signal)$$

`[-0.3, 0.2, 0.1]` is saying:

Happy: you predicted 0.7, should have been 1.0 — you're 0.3 short, so negative blame (push this up)

Sad: you predicted 0.2, should have been 0.0 — you overclaimed, positive blame (push this down)

Angry: same, you overclaimed by 0.1

It goes all the way, until it reaches our CNN architecture, and then undoes all the pooling, flattening, until it reaches the input layer.

Building the Network

In this part, we are building the ANN from scratch using only Numpy.

Forward Pass

```
self.filter_size = 3
self.num_filters = 32
self.conv_filters = np.random.randn(self.num_filters, self.filter_size, self.filter_size) * 0.01
# This is our convolutional layer, for our CNN. Why is it 32, 3 3?
# We want 32 filters sliding accross an image, 3*3 in size.
# this creates exactly that, a list of 32 values, in 3x3 formats, with random
# initialization. This is the filter idea
```

Filters for our convolutional layer

```
self.conv_biases = np.random.randn(32) * 0.01
# This is the bias for each filter, we have 32 filters, so we have 32 biases. The weighted sum
# will apply for each filter, so we need a bias for each one aswell
```

Biases

```
self.max_pool_size = 2
# Max pooling is a downsampling technique, it reduces the spatial dimensions of the input.
# It does this by taking the maximum value from a set of values in a defined window (in this case, 2x2).
# This helps to reduce the computational complexity and also helps
# to make the model more robust to small translations in the input.
```

Max Pool size (2x2)

```
self.pos_size = (image_size - 3 + 1) // 2
# After the convolutional layer, the spatial dimensions of the output will be reduced.
# The formula for calculating the output size after convolution is:
# output_size = (input_size - filter_size + 2 * padding) / stride + 1
# In our case, we have an input size of 48, a filter size of 3, no padding, and a stride of 1.
# So the output size after convolution will be:
# output_size = (48 - 3 + 0) / 1 + 1 = 46
# After max pooling with a pool size of 2, the spatial dimensions will be halved, so the final output siz
# pos_size = 46 // 2 = 23
```

Size after con. Layer and pooling

```python
# First layer's weights & biases
self.W1 = np.random.randn((self.pos_size * self.pos_size * self.num_filters), self.hidden_size) * 0.01
self.b1 = np.zeros(self.hidden_size)

# Second layer's weights & biases
self.W2 = np.random.randn(128, 5) * 0.01
self.b2 = np.zeros(5)
```

Random weight init. And empty bias init.

```python
def conv_forward(self, x):

    size_after = self.image_size - 3 + 1
    output_array = np.zeros((self.num_filters, size_after, size_after))

    for index, filter in enumerate(self.conv_filters):
        for col in range(size_after):
            for row in range(size_after):
                patch = x[col:col+3, row:row+3]
                output_array[index, row, col] = np.sum(patch * filter) + self.conv_biases[index]

    return output_array
```

Conv layer forward pass,
Slicing the image into 3x3, weighted sum + bias (initiated bias as 0)

```python
def relu(self, x):
    return np.maximum(0, x)
```

ReLU activation function

```python
    def maxpool_forward(self, x):

        size_after = (self.image_size - 3 + 1) // 2
        output_array = np.zeros((self.num_filters, size_after, size_after))

        for index in enumerate(self.conv_filters):
            for col in range(size_after):
                for row in range(size_after):
                    patch = x[index, row * 2:row * 2 + 2, col * 2:col*2+2]
                    output_array[index, row, col] = np.max(patch)

        return output_array
```

Max pooling [2:2], returning max value from those patches, diminishing to 23x23

```python
def conv_forward(self, x):

    size_after = self.image_size - 3 + 1
    output_array = np.zeros((self.num_filters, size_after, size_after))

    for index, filter in enumerate(self.conv_filters):
        for col in range(size_after):
            for row in range(size_after):
                patch = x[row:row+3, col:col+3]
                output_array[index, row, col] = np.sum(patch * filter) + self.conv_biases[index]

    return output_array
```

Convolutional layer, applying the filter solution

```python
def flatten(self, x):
    return x.reshape(-1)
```

Flattening our convolutional layer into a 1D list

```python
def fc_forward(self, x, W, b):
    return x @ W + b
```

Weighted sum + bias

```python
def forward(self, x):

    self.input = x
    # save x as the input

    result = self.conv_forward(x)
    result = self.relu(result)
    self.conv_output = result
    # save conv layer + relu output

    result = self.maxpool_forward(result)
    self.pool_shape = result.shape

    result = self.flatten(result)
    self.flat = result
    # store flatten

    result = self.fc_forward(result, self.W1, self.b1)
    result = self.relu(result)
    self.fc1_out = result
    # store fc1 output

    result = self.fc_forward(result, self.W2, self.b2)
    result = self.softmax(result)


    return result
```

Entire flow of processing, storing the results along the way so we can access them during backpropagation

Back Propagation

1.

```
def backward(self, predicted, true):

    # true is our OHE label for the true value of whatever emotion we were trying to detect
    # [0, 0, 1, 0, 0], for example

    # predicted is what our model's predictons
    # [0.1, -0.3, 0.5, 0.2, -0.7], for example
```

Back propagation flow pt. 1, def.

```
dL = predicted - true
# dL is the loss function, predicted - true
```

Backpropagation flow pt 2. Loss function

```
dW2 = self.fc1_out.reshape(-1,1) @ dL.reshape(1,-1)
# @ is the symbol for matrix multiplication, and matrix multiplication requires the shape to match
# the shape of fc1_out is (128,) (128 neurons in the hidden layer) the shape of dL (which is our probabilities is 5,)
# so doing this reshape, we make fc1_out (128, 1) and dL (1, 5), so the result of the matrix multiplication is (128, 5)
# which is the shape of W2
```

Backpropagation flow pt 3. Reshaping weights and multiplying

```
db2 = dL
# db2 (hidden layer 2's bias) is the value of dL which is the loss function
# why do we ues it as a bias? because during backprop, we ask, "what caused this error?"; the bias contributed to the error
# by exactly the amount of the error itself, because it's a flat addition with no multiplication~
```

Backpropagation flow pt 4. Assigning bias

```
dL = dL @ self.W2.T
# To unpack; .T means transpose, in the forward pass we went 128 -> 5, in the backward pass we go 5- -> 128
# W2 (3, 2):          W2.T (2, 3):
# [[0.5, 0.1],        [[0.5, 0.3, 0.2],
#  [0.3, 0.8],   →     [0.1, 0.8, 0.4]]
#  [0.2, 0.4]]      You, 15 minutes ago • Uncommitted changes
# rows become columns, columns become rows

# dL @ W2.T:
# [-0.3, 0.6] @ [[0.5, 0.3, 0.2],
#                [0.1, 0.8, 0.4]]
#
# neuron 0: (-0.3 × 0.5) + (0.6 × 0.1) = -0.15 + 0.06 = -0.09
# neuron 1: (-0.3 × 0.3) + (0.6 × 0.8) = -0.09 + 0.48 =  0.39
# neuron 2: (-0.3 × 0.2) + (0.6 × 0.4) = -0.06 + 0.24 =  0.18
#
# result = [-0.09, 0.39, 0.18]
# each neuron receives blame proportional to its weight connections
```

Back propagation flow pt 5, Going back another layer

```
dL = dL * (self.fc1_out > 0)
# relu backpropagation, multiplies the loss by 1 if the output of the relu was greater than 0,~
#  and by 0 if it was less than or equal to 0.
```

Back propagation flow pt 6, ReLU

```
dW1 = self.flat.reshape(-1,1) @ dL.reshape(1,-1)
db1 = dL
dL = dL @ self.W1.T
# same as dW2
```

Back propagation flow pt 7, next hidden layer

```
# de-pool
conv = self.conv_output
# get our conv layer's output
empty_conv = np.zeros(conv.shape)
# create an empty conv we'll fill in

for index in range(self.num_filters):
    for row in range(23):
        for col in range(23):
            empty_conv[index, row*2:row*2+2, col*2:col*2+2] += dL[index, row, col] * self.max_mask[index, row*2:row*2+2, col*2:col*2+2]
# loop over the maxpool output (filter by filter, row by row, col for col)
# what we do here is simple; when we max pooled, we only got 1 single value from each 2x2
# so in this case, we' have a max__mask that has 1s where the max values were, and 0s everywhere else.
#  So to de-pool, we take the gradient from the maxpool layer (dL[index, row, col])
# and we put it back in the position of the max value in the 2x2 patch, which is what
# self.max_mask is for. We multiply dL by the max_mask to ensure that
# only the position of the max value gets the gradient, and the rest get 0.
```

Back propagation flow pt 8, de-pooling

```
dL = empty_conv
dL = dL * (self.conv_output > 0)

# run ReLU again and set dL to our conv layer's output where it's active, and 0 where it's not
```

Back propagation flow pt 9, ReLU

```
dW_conv = np.zeros_like(self.conv_filters)
db_conv = np.zeros_like(self.conv_biases)

# create empty arrays for the gradients of the convolutional filters and biases

size_after = self.image_size - self.filter_size + 1

for index, filter in enumerate(self.conv_filters):
    for col in range(size_after):
        for row in range(size_after):
            patch = self.input[row:row+3, col:col+3]
            dW_conv[index] += patch * dL[index, row, col]
            db_conv[index] += dL[index, row, col]
# loop over the conv layer's output (filter by filter, row by row, col for col)
# for each position we get the corresponding patch and do the exact thing we did above
# the gradient for the filter is the patch multiplied by the gradient coming from above (dL[index, row, col])
# the gradient for the bias is just the gradient coming from above (dL[index, row, col]) because ~
# the bias is added flatly to the output of the convolution, so its contribution to
# the error is directly proportional to the error itself
```

Back propagation flow pt 10, adding the gradients to our input

Training Loop

```python
def train(cnn, X_train, y_train, epochs, lr=0.01, batch_size=32):
    for epoch in range(epochs):
        indices = np.random.permutation(len(X_train))
        epoch_loss = 0

        # split into batches
        for i in range(0, len(X_train), batch_size):
            batch_indices = indices[i:i+batch_size]
            X_batch = X_train[batch_indices]
            y_batch = y_train[batch_indices]

            dW_conv_acc = np.zeros_like(cnn.conv_filters)
            db_conv_acc = np.zeros_like(cnn.conv_biases)
            dW1_acc = np.zeros_like(cnn.W1)
            db1_acc = np.zeros_like(cnn.b1)
            dW2_acc = np.zeros_like(cnn.W2)
            db2_acc = np.zeros_like(cnn.b2)

            if i % (batch_size * 2) == 0:
                    print(f"  Batch {i//batch_size}/{len(X_train)//batch_size}")

            for image, label in zip(X_batch, y_batch):

                predicted = cnn.forward(image)
                loss = -np.log(predicted[np.argmax(label)] + 1e-8)
                dW_conv, db_conv, dW1, db1, dW2, db2 = cnn.backward(predicted, label)
                dW_conv_acc += dW_conv
                db_conv_acc += db_conv
                dW1_acc += dW1
                db1_acc += db1
                dW2_acc += dW2
                db2_acc += db2
                epoch_loss += loss
            cnn.update(dW_conv_acc/batch_size, db_conv_acc/batch_size,
                dW1_acc/batch_size, db1_acc/batch_size,
                dW2_acc/batch_size, db2_acc/batch_size, lr)
        print(f"Epoch {epoch+1}/{epochs} | Loss: {epoch_loss/len(X_train):.4f}")
```

The Numpy Problem

1. **From theory to implementation: why not plain Python?**

At this point we understand the theory — layers, neurons, weights, activation functions, forward pass, loss, and backpropagation. In principle, all of this could be implemented using only Python and NumPy. And for learning purposes, that is actually a valid approach.

However, there is a fundamental problem with that path: **performance**.

Every operation we described — multiplying weights, summing inputs, computing gradients — needs to happen across thousands of neurons, for thousands of training examples, repeated over hundreds of training cycles. In plain Python, each of these operations would be a loop. Loops in Python are slow. For anything beyond a toy example, a pure Python or even a pure NumPy implementation becomes impractical.

This is where **PyTorch** comes in. PyTorch is a Python library that lets you build and train neural networks without implementing the math yourself from scratch. Under the hood, it runs the same operations we described — but written in optimized C++, with optional GPU acceleration. The same math runs orders of magnitude faster.

The other major advantage of PyTorch is **automatic differentiation**. Instead of manually deriving and coding the backpropagation equations, PyTorch tracks every operation and computes gradients automatically. This removes an entire layer of complexity.

Pytorch Implementation

```python
import torch
import torch.nn as nn
import torch.optim
import numpy as np
```

Pytorch pt 1. Imports

```python
class CNN(nn.Module):

    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3)   # 1 grayscale input, 32 filters, 3x3 kernel
        self.pool = nn.MaxPool2d(2, 2)      # halves spatial dims
        self.fc1 = nn.Linear(32*23*23, 128) # flatten → 128 neurons
        self.fc2 = nn.Linear(128, 5)        # 128 → 5 emotion classes

    def forward(self, x):
        x = torch.relu(self.conv1(x))   # conv + activate
        x = self.pool(x)                # downsample: 46x46 → 23x23
        x = x.view(x.size(0), -1)       # flatten, keeping batch dim
        x = torch.relu(self.fc1(x))     # FC + activate
        x = self.fc2(x)                 # raw logits — no softmax (CrossEntropyLoss handles it)
        return x
```

Pytorch pt 2. CNN class + forward pass

```python
class LoadData(torch.utils.data.Dataset):

    def __init__(self, split):
        super().__init__()
        data = np.load('dataset.npz')

        # pick the right split
        if split == 'train':
            self.X = data['X_train']
            self.y = data['y_train']
        elif split == 'val':
            self.X = data['X_val']
            self.y = data['y_val']
        elif split == 'test':
            self.X = data['X_test']
            self.y = data['y_test']

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        image = (torch.tensor(self.X[idx]).float() / 255).unsqueeze(0)  # [0,255] → [0,1], add channel dim
        label = torch.argmax(torch.tensor(self.y[idx]).float()).long()   # OHE → class index
        return image, label
```

Pytorch pt 3. Load data class

```python
train_dataset = LoadData('train')
val_dataset   = LoadData('val')
test_dataset  = LoadData('test')

# shuffle train so batches aren't always the same order; val/test order doesn't matter
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = torch.utils.data.DataLoader(val_dataset,   batch_size=32, shuffle=False)
test_loader  = torch.utils.data.DataLoader(test_dataset,  batch_size=32, shuffle=False)

model     = CNN()
loss_fn   = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
 # Adam: adaptive gradient descent — adjusts the lr per weight automatically
```

Pytorch pt 4. Load train/val/test data into a loader (which separates into batches and/or shuffles), Load up our model,
Load up an optimizer for the gradient descent

```python
best_accuracy = 0

for i in range(50):
    train_loss = 0
    val_loss   = 0
    correct    = 0
    total      = 0

    # ── train ──────────────────────────────────────
    model.train()
    for image, label in train_loader:
        optimizer.zero_grad()           # clear last batch's gradients
        x = model(image)
        loss = loss_fn(x, label)
        loss.backward()                 # compute gradients
        optimizer.step()                # update weights
        train_loss += loss.item()

    # ── validate ───────────────────────────────────
    model.eval()
    with torch.no_grad():               # no gradients needed — faster + less memory
        for image, label in val_loader:
            x           = model(image)
            loss        = loss_fn(x, label)
            predictions = torch.argmax(x, dim=1)    # highest logit = predicted class
            correct     += (predictions == label).sum().item()
            total       += label.size(0)
            val_loss    += loss.item()

    # ── log ────────────────────────────────────────
    train_loss /= len(train_loader)
    val_loss   /= len(val_loader)
    accuracy    = correct / total

    if accuracy > best_accuracy:
        best_accuracy = accuracy
        torch.save(model.state_dict(), 'best_model.pth')    # save weights only, not full model

    print(f'Epoch {i+1:>2} | Train Loss: {train_loss:.4f}  Val Loss: {val_loss:.4f}  Acc: {accuracy:.4f}')
```

Pytorch pt 5. Training loop, storing the loss & accuracy to see if our model is still learning, Storing the best model's weights

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
model     = CNN().to(device)
```

```
image, label = image.to(device), label.to(device)
```

Pytorch pt 6. These lines tell pytroch to run training on our GPU instead of CPU

```
Epoch 11 | Train Loss: 1.3461  Val Loss: 1.3451  Acc: 0.4354
Epoch 12 | Train Loss: 1.3374  Val Loss: 1.3650  Acc: 0.4276
Epoch 13 | Train Loss: 1.3312  Val Loss: 1.3500  Acc: 0.4380
Epoch 14 | Train Loss: 1.3252  Val Loss: 1.3273  Acc: 0.4425
Epoch 15 | Train Loss: 1.3187  Val Loss: 1.3296  Acc: 0.4400
Epoch 16 | Train Loss: 1.3144  Val Loss: 1.3301  Acc: 0.4436
Epoch 17 | Train Loss: 1.3105  Val Loss: 1.3170  Acc: 0.4493
Epoch 18 | Train Loss: 1.3061  Val Loss: 1.3162  Acc: 0.4526
Epoch 19 | Train Loss: 1.3009  Val Loss: 1.3123  Acc: 0.4510
Epoch 20 | Train Loss: 1.2973  Val Loss: 1.3110  Acc: 0.4483
Epoch 21 | Train Loss: 1.2941  Val Loss: 1.3094  Acc: 0.4507
Epoch 22 | Train Loss: 1.2913  Val Loss: 1.3002  Acc: 0.4514
Epoch 23 | Train Loss: 1.2863  Val Loss: 1.3174  Acc: 0.4450
Epoch 24 | Train Loss: 1.2841  Val Loss: 1.3028  Acc: 0.4577
Epoch 25 | Train Loss: 1.2807  Val Loss: 1.3045  Acc: 0.4546
Epoch 26 | Train Loss: 1.2770  Val Loss: 1.2929  Acc: 0.4633
Epoch 27 | Train Loss: 1.2729  Val Loss: 1.2970  Acc: 0.4528
Epoch 28 | Train Loss: 1.2696  Val Loss: 1.2862  Acc: 0.4609
Epoch 29 | Train Loss: 1.2646  Val Loss: 1.2835  Acc: 0.4635
Epoch 30 | Train Loss: 1.2592  Val Loss: 1.2820  Acc: 0.4614
Epoch 31 | Train Loss: 1.2548  Val Loss: 1.2945  Acc: 0.4570
```

Pytorch pt 7. Model stopped getting better, val loss and train loss similar (overfitting)

```
Epoch  1 | Train Loss: 1.5707  Val Loss: 1.5691  Acc: 0.3066
Epoch  2 | Train Loss: 1.5688  Val Loss: 1.5657  Acc: 0.3066
Epoch  3 | Train Loss: 1.4865  Val Loss: 1.4134  Acc: 0.4088
Epoch  4 | Train Loss: 1.4012  Val Loss: 1.3905  Acc: 0.4194
Epoch  5 | Train Loss: 1.3812  Val Loss: 1.3745  Acc: 0.4248
Epoch  6 | Train Loss: 1.3624  Val Loss: 1.3557  Acc: 0.4338
Epoch  7 | Train Loss: 1.3436  Val Loss: 1.3336  Acc: 0.4507
Epoch  8 | Train Loss: 1.3208  Val Loss: 1.3142  Acc: 0.4576
Epoch  9 | Train Loss: 1.2954  Val Loss: 1.2915  Acc: 0.4665
Epoch 10 | Train Loss: 1.2702  Val Loss: 1.2732  Acc: 0.4693
Epoch 11 | Train Loss: 1.2485  Val Loss: 1.2675  Acc: 0.4674
Epoch 12 | Train Loss: 1.2291  Val Loss: 1.2452  Acc: 0.4841
Epoch 13 | Train Loss: 1.2123  Val Loss: 1.2456  Acc: 0.4859
Epoch 14 | Train Loss: 1.1987  Val Loss: 1.2503  Acc: 0.4837
Epoch 15 | Train Loss: 1.1846  Val Loss: 1.2172  Acc: 0.4949
```

```
self.conv2 = nn.Conv2d(32, 64, 3)  # 32 input channels, 64 filters, 3x3 kernel
self.pool = nn.MaxPool2d(2, 2)      # halves spatial dims
self.fc1 = nn.Linear(10*10*64, 128) # flatten → 128 neurons
```

```
x = torch.relu(self.conv2(x))    # conv 2 + activate [64 * 10 * 10]
x = self.pool(x)
```

Pytorch pt 8. Introduction of a second conv layer

```
if self.augment:
    transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    ])
    image = transform(image)
```

```
import torchvision.transforms as transforms
```

Pytorch pt 9. Data augmentation

Conclusion

This document set out to understand, end-to-end, how a neural network works — not by using a framework as a black box, but by building one from the ground up.

Starting from a single neuron, we traced the full path: how raw pixel values become weighted sums, how activation functions introduce non-linearity, how a loss function quantifies the network's mistakes, and how backpropagation distributes blame backwards through every layer to correct those mistakes.

From there, we moved into the architecture that makes image recognition practical — the Convolutional Neural Network. Filters, feature maps, pooling, flattening, and stacking layers into something capable of recognizing not just pixels, but patterns within patterns.

We then implemented all of it manually in NumPy — not because it is the practical choice, but because there is no better way to understand something than to build it yourself. Every forward pass written by hand, every gradient computed explicitly.

Finally, we transitioned to PyTorch — not as a shortcut, but as the natural next step once the foundations are solid. The math is identical. The difference is that PyTorch handles it faster, cleaner, and at scale.

What this project demonstrated:

1. A neural network is not magic — it is repeated, structured arithmetic guided by a signal of how wrong it currently is
2. The architecture matters: CNNs exist because MLPs are blind to spatial relationships in images
3. Understanding the NumPy implementation makes the PyTorch code readable rather than mysterious

Thank you for reading,

By okupa